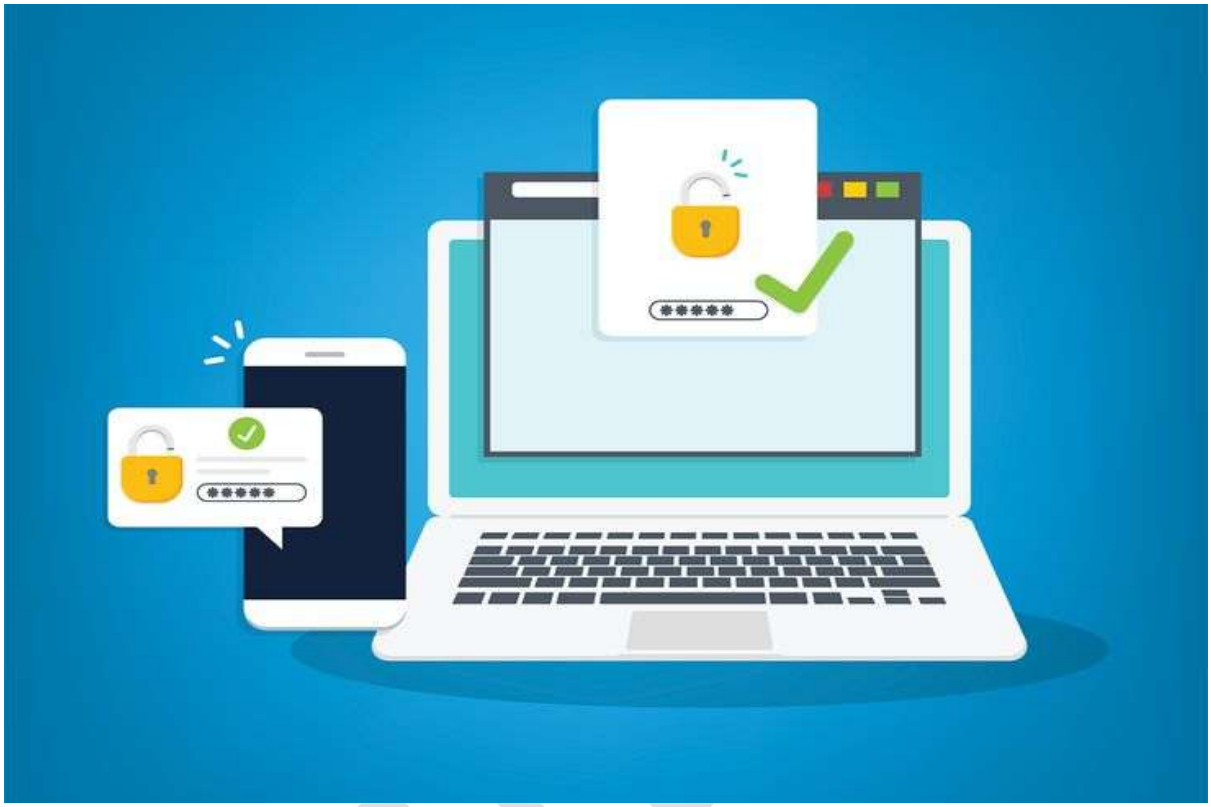
Hashing

About hashing functions and their uses in password verification and file integrity checking.



Hashing is a fundamental cryptographic technique used to ensure data integrity and secure authentication by converting data of any size into a fixed-length hash value that cannot be reversed. It plays a critical role in verifying file integrity, securely storing passwords, and detecting data tampering through mechanisms like hashing algorithms, salting, and HMAC. This module covers core hashing concepts, collision risks, secure password storage practices, hash identification, and practical cracking techniques using tools like Hashcat and John the Ripper. It also highlights real-world applications such as integrity verification of downloaded files and secure authentication systems, with hands-on command-line examples and attack methodologies inspired by practical cybersecurity workflows.

This documentation and learning approach is strongly inspired by the structured, hands-on content provided by TryHackMe, which significantly contributed to practical understanding of hashing concepts.

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Hash Functions – Concept + Formula

Definition

A hash function is defined as:

$$H(x) = y$$

Where:

- H= Hash Function
- x= Input (any size data)
- y= Output (fixed-size hash value)

Important Full Forms

- **MD5** – Message Digest Algorithm 5
- **SHA** – Secure Hash Algorithm
- **SHA-1** – Secure Hash Algorithm 1
- **SHA-256** – Secure Hash Algorithm 256-bit
- **ASCII** – American Standard Code for Information Interchange

Core Properties

1. Deterministic

Same input → Same output

2. Fixed Length Output

Algorithm	Output Size
MD5	128 bits (16 bytes)
SHA-1	160 bits (20 bytes)
SHA-256	256 bits (32 bytes)

WEIRDO

3. Avalanche Effect

A small change in input causes a major change in output.

Example:

```
echo "T" | sha256sum
echo "U" | sha256sum
```

Even **1-bit change → completely different hash**

4. One-Way Function

Given H(x), finding x is computationally infeasible

Command Usage

```
md5sum file.txt
sha1sum file.txt
sha256sum file.txt
```

Output format:

<hash_value> <filename>

Hexadecimal Encoding Logic

- 1 Byte = 8 bits
- 1 Hex digit = 4 bits

$1\ b\ yte = 2\ hex\ digits$

Example:

- MD5 → 16 bytes → 32 hex characters
- SHA256 → 32 bytes → 64 hex characters

2. Hash Collisions – Mathematical Explanation

WEIRDO

Definition

A collision occurs when:

$$H(x_1) = H(x_2), x_1 \neq x_2$$

Why Collisions Occur

Due to the **Pigeonhole Principle**:

$$\text{Inputs} > \text{Outputs} \rightarrow \text{collisions unavoidable}$$

Example:

- 8-bit hash → $2^8 = 256$outputs

Security Insight

- MD5 → Broken
- SHA-1 → Broken
- SHA-256 → Secure

3. Password Storage – Security Logic

Insecure Methods

1. Plaintext Storage

Stores actual password → High risk

2. Encryption Storage

Problem:

- Requires storing key
- If key leaks → all passwords exposed

3. Weak Hashing

- Fast → easy to brute-force
- No salt → vulnerable

4. Secure Password Storage

WEIRDO

Formula

$$Hash = H(password + salt)$$

Full Forms

- **PBKDF2** – Password-Based Key Derivation Function 2
- **bcrypt** – Blowfish-based cryptographic hashing
- **scrypt** – Memory-hard key derivation function
- **Argon2** – Modern password hashing algorithm

Example

Password: AL4RMc10k
Salt: Y4UV*^(=go_!

Combined:
AL4RMc10kY4UV*^(=go_!

Hash:

echo -n "AL4RMc10kY4UV*^(=go_!" | sha256sum

Why Salt is Important

Without salt:

password → same hash

With salt:

password + salt → unique hash

WEIRDO

5. Rainbow Tables

Definition

Precomputed mapping:

$$Hash \rightarrow Plaintext$$

Example:

e10adc3949ba59abbe56e057f20f883e → 123456\

Defense

$Use\ u$ $nique\ s$ alt

6. Hash Identification

Linux Shadow File Format

$username:$prefix$options$salt$hash

Prefix Meaning

Prefix	Algorithm
y	yescrypt
$2b$	bcrypt
6	SHA-512
1	MD5

Full Forms

- **NTLM** – New Technology LAN Manager
- **SAM** – Security Accounts Manager

Windows Password Storage

- Stored in SAM database
- Uses NTLM hashing

7. Password Cracking – Logic + Commands

WEIRDO

Core Logic

$$Guess \rightarrow Hash \rightarrow Compare$$

If:

$$H(guess) = target_hash$$

→ Password found

Tools Full Forms

- **GPU** – Graphics Processing Unit
- **CPU** – Central Processing Unit

Hashcat Command

hashcat -m <hash_type> -a <attack_mode> hashfile wordlist

Parameters

- **-m** → hash type
- **-a** → attack mode

Example

hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

Attack Types

- Dictionary Attack → uses wordlist
- Brute Force Attack → tries all combinations

WEIRDO

8. Integrity Checking

Formula

$$If\ H(file_1) = H(file_2) \rightarrow files\ are\ identical$$

Command

sha256sum file.iso

Compare with official hash.

9. HMAC (Hash-Based Message Authentication Code)

Full Form

- **HMAC** – Hash-Based Message Authentication Code

Formula

$$HMAC(K, M) = H((K \oplus opad) \mid\mid H((K \oplus ipad) \mid\mid M))$$

Where:

- K= Secret Key
- M= Message
- $\oplus$= XOR (Exclusive OR)
- $\mid\mid$= Concatenation

Purpose

- Integrity
- Authentication

10. Hashing vs Encoding vs Encryption

WEIRDO

Feature	Hashing	Encoding	Encryption
Reversible	No	Yes	Yes
Uses Key	No	No	Yes
Purpose	Integrity	Format	Confidentiality

Encoding Full Forms

- **UTF** – Unicode Transformation Format
- **Base64** – Binary-to-text encoding scheme

Base64 Example

echo "TryHackMe" | base64

Output:

VHJ5SGFja01lCg==

Decode:

echo "VHJ5SGFja01lCg==" | base64 -d

11. Final Key Takeaways

- Hashing is **one-way and irreversible**
- Used for:
  - Password security
  - Data integrity
- Salt prevents rainbow table attacks
- Strong hashing = slow + secure
- Cracking = guessing + hashing + comparing
- HMAC adds authentication with a secret key

12. Practical Command Cheat Sheet

WEIRDO

```
# Generate hash
md5sum file
sha256sum file

# Identify hash
hashid hash.txt

# Crack hash
hashcat -m 0 -a 0 hash.txt rockyou.txt
john hash.txt --wordlist=rockyou.txt

# Encode/Decode
base64
base64 -d
```

WEIRDO